AWS Architecture & Deployment Options

Executive Summary & Benchmark Report

Author: Tran Hai Dang

Role: Cloud / DevOps Engineer Intern Candidate

Institution: Hanoi University of Civil Engineering (HUCE), Class 69PM3

Contact: tranhaidang.devops@gmail.com

AWS account & region: 8258****4433 | Singapore (ap-southeast-1)

Date: October 2026

Project repository: https://github.com/TranHaiDang-DevOps-25112006/CloudEvo-Benchmark

GitHub profile: https://github.com/TranHaiDang-DevOps-25112006/TranHaiDang-DevOps-25112006

Note on identifiers and lab resources. This report deliberately keeps real lab identifiers (public IP addresses, instance, security-group and RDS identifiers, container hostnames) so that every figure can be traced to the logs and screenshots in the repository. These are lab resources created only for this take-home assignment. They were deleted right after the logs were captured: Terraform-managed resources with `terraform destroy`, manually created ones by hand. None of them exist on AWS any more. In a real production environment, identifiers like these should be masked or replaced with placeholders, in line with information-security practices such as ISO/IEC 27001, to limit infrastructure reconnaissance. The AWS account ID outlives any single resource, so it is partially masked.

1. Executive Summary

This report compares four ways of hosting the same Node.js/Express order-tracker web application on AWS in the Singapore region (ap-southeast-1). Each architecture is evaluated against the three questions raised by the customer: **(1)** how long does it take to host the application, **(2)** how much does it cost per month, and **(3)** how long does recovery take after an outage (RTO) and how much data is lost (RPO).

How to read the numbers. Deployment times for manual (ClickOps) and EC2 User Data builds, and every recovery time, were measured on live AWS resources (one run per test; recovery times were captured by `scripts/dr_monitor.py`, which polls `/health` every second). Monthly costs are calculated from on-demand prices for ap-southeast-1 at 730 hours per month (Section 4). Terraform was applied and timed end to end for Scenario 1 only; Scenarios 2–4 are codified but not yet timed (Section 7). Values tagged **ESTIMATE** are design estimates, not measurements.

Table 1 — Summary across the four scenarios

	Scenario 1 Stateless web on one EC2	Scenario 2 Web + local PostgreSQL on one EC2	Scenario 3 Web EC2 + dedicated DB EC2	Scenario 4 Web EC2 + Amazon RDS
Deployment: manual (ClickOps)	20 min	35 min	50 min	45 min
Deployment: EC2 User Data	2m 48s (−86.0%)	3m 15s (−90.7%)	~5m 00s (−90.0%)	~11m 00s (−75.6%), incl. RDS creation 8m 30s
Deployment: Terraform	~1m 45s (−91%)	**NOT MEASURED**	**NOT MEASURED**	**NOT MEASURED**
Monthly cost (on-demand, ap-southeast-1)	$15.21	$16.55	$28.10	$36.22 Single-AZ (deployed) $57.96 Multi-AZ (priced only)
Recovery time (RTO), measured	449.48 s: instance terminated, replaced manually	34.96 s: DB CPU/RAM saturation 82.49 s: containers + volume wiped, restored from S3	86.05 s: web tier failure 44.07 s: DB restored from S3	11.46 s: RDS reboot (Single-AZ) PITR restore ready in ~8–10 min
Data loss observed (RPO)	100% (in-memory state)	Up to 1 h (Order #103 lost)	Web failure: 0 DB failure: up to 1 h	0 on RDS reboot PITR: < 5 min per AWS design (**ESTIMATE**)

Key takeaways

- **Automation drives delivery speed.** EC2 User Data cut hand-built deployments by 75.6–90.7%, and Terraform brought the full Scenario 1 stack (VPC, security groups, IAM, EC2) to about 1m 45s.
- **Cost rises with isolation and managed services**, from $15.21 (Scenario 1) to $36.22 (Scenario 4, Single-AZ). Avoiding a NAT Gateway ($43.07/month before data processing) keeps the private-subnet designs affordable.
- **Managed RDS recovered fastest and lost no data**: 11.46 s with automatic reconnection, versus 44.07–82.49 s for restore-from-S3 designs that can lose up to an hour of data. Keeping state on the web host (Scenarios 1–2) is the most fragile choice.

2. Architecture Scenarios

All scenarios share one custom VPC (**cloudevo-vpc**, 10.0.0.0/16) with two public subnets (10.0.1.0/24 in ap-southeast-1a, 10.0.2.0/24 in 1b), two private subnets (10.0.11.0/24 in 1a, 10.0.12.0/24 in 1b), an Internet Gateway (cloudevo-igw) and an S3 Gateway VPC Endpoint (cloudevo-vpce-s3) that carries backup traffic to Amazon S3 at no charge. No NAT Gateway is used. Compute runs in ap-southeast-1a; the second AZ is part of the network design and is required by the RDS DB subnet group.

Scenario 1: Single web application on a single EC2 (stateless, in-memory)

At a glance | Deploy: 20 min (ClickOps) / 2m 48s (User Data) / ~1m 45s (Terraform) | Cost: $15.21/month | Measured RTO: 449.48 s (manual replacement) | Data loss: 100% of in-memory state | Best for: stateless front ends and demos

Source code: scenarios/case1-single-ec2-web | **Experiment report:** docs/Day1

- **Compute:** 1× EC2 t3.micro (2 vCPU, 1 GiB RAM, Ubuntu 22.04 LTS), 20 GB gp3 root volume, public subnet 10.0.1.0/24. Source: `scenarios/case1-single-ec2-web/`, `terraform/` (scenario_case=1).
- **Application:** Docker with Nginx reverse proxy (port 80) forwarding to the Node.js/Express Order Tracker (port 3000), `STORAGE_MODE=memory`: all state lives in process memory.
- **Security and probing:** security group cloudevo-web-sg (sg-03c49016a8a6f5ecd) allows inbound TCP 80 and 22; CloudWatch basic metrics ($0); `/health` returns status, mode, database state, latency and hostname.
- **Evidence:** public IPv4 47.129.189.162 (User Data build) and 54.254.173.180 (Terraform build); container hostname fe5dc1f64f31 before and 363732ec3ca3 after recovery.

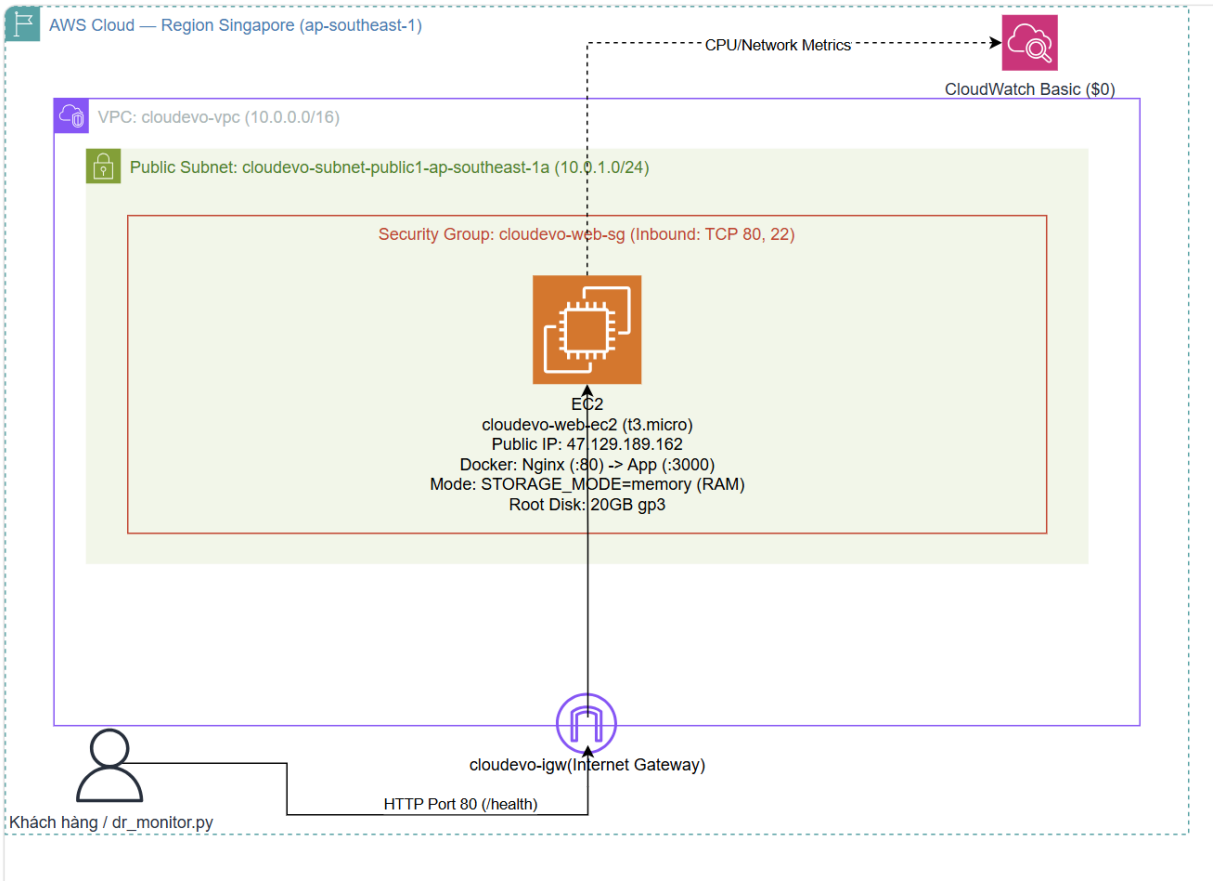


Figure 1.1: Scenario 1 — stateless single EC2 (t3.micro, STORAGE_MODE=memory) in Public Subnet 1 (10.0.1.0/24).

Scenario 2: Web application + local PostgreSQL on a single EC2 (stateful monolith)

At a glance | Deploy: 35 min (ClickOps) / 3m 15s (User Data) / Terraform not measured | Cost: $16.55/month | Measured RTO: 34.96 s (DB saturation), 82.49 s (restore from S3) | Data loss: up to 1 h | Best for: internal tools and MVPs that accept up to 1 h of data loss

Source code: scenarios/case2-single-ec2-web-db | **Experiment report:** docs/Day2

- **Compute:** 1× EC2 t3.micro, 25 GB gp3, public subnet 10.0.1.0/24, three containers orchestrated by Docker Compose: cloudevo-nginx (80), cloudevo-web (3000, STORAGE_MODE=postgres) and cloudevo-postgres (PostgreSQL 15 Alpine, published only on loopback 127.0.0.1:5432).
- **No stored AWS credentials:** the instance assumes IAM role cloudevo-ec2-s3-backup-role through an instance profile; its customer-managed policy cloudevo-ec2-s3-backup-policy grants access only to bucket cloudevo-backup-dangth.

- **Backup:** an hourly cron job runs `backup_s3.sh` (pg_dump piped to gzip) and uploads to S3 through the Gateway Endpoint (about 4 s).
- **Evidence:** public IPv4 46.137.231.237 (private 10.0.1.14); hostnames 75da0efaeb9f (Test 2A), 5ede407bab53 (Test 2B), 9db872eb2c3f → fa46b20a328f (RPO check).

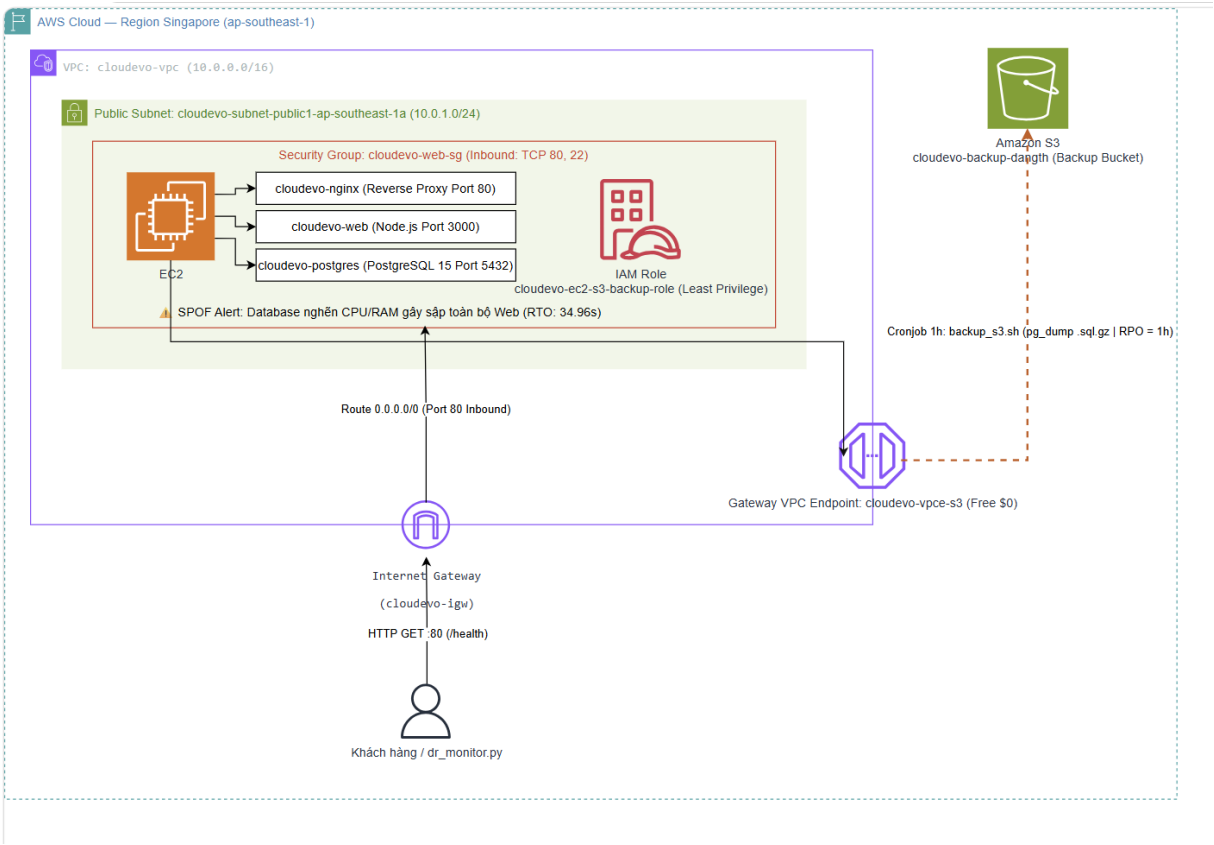


Figure 1.2: Scenario 2 — monolith web + PostgreSQL 15 on one EC2 with least-privilege S3 backup through the VPC endpoint.

```
# IAM policy: cloudevo-ec2-s3-backup-policy
{ "Version": "2012-10-17",
  "Statement": [
    { "Sid": "AllowListCloudEvoBucket", "Effect": "Allow",
      "Action": ["s3:ListBucket"], "Resource": "arn:aws:s3:::cloudevo-backup-dangth" },
    { "Sid": "AllowReadWriteBackupFiles", "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::cloudevo-backup-dangth/*" } ] }
```

Scenario 3: Web on EC2 + dedicated database on a second EC2 (2-tier decoupled)

At a glance | Deploy: 50 min (ClickOps) / ~5m 00s (User Data) / Terraform not measured | Cost: $28.10/month | Measured RTO: 86.05 s (web tier), 44.07 s (DB restore from S3) | Data loss: web 0, DB up to 1 h | Best for: scaling or tuning web and database separately

Source code: scenarios/case3-two-ec2-web-db | **Experiment report:** docs/Day3

- **Web tier** (cloudevo-web-ec2, i-035bc597ce46ffb41): EC2 t3.micro, 20 GB gp3, public subnet 10.0.1.0/24 (subnet-0914b2264a767ee33), public IPv4 13.229.225.174.
- **Database tier** (cloudevo-db-ec2, i-0e14eeeabd149cc1f): EC2 t3.micro, 20 GB gp3, private subnet 10.0.11.0/24 (subnet-07a286c5fddc36e80), private IP 10.0.11.246, no public IP. PostgreSQL 15 uses scram-sha-256 authentication; measured query latency 12 ms.
- **Security-group chaining:** cloudevo-db-sg (sg-07212bbabd8d70f2c) accepts TCP 5432 (rule sgr-008a6022822e3db70) and TCP 22 (rule sgr-0986bc76a72d7a532) only from the web security group sg-03c49016a8a6f5ecd, never from a CIDR range. The web EC2 doubles as the SSH bastion.
- **No NAT Gateway:** the DB host needed outbound Internet access only during its first boot (to install PostgreSQL); afterwards hourly `pg_dump` backups reach S3 privately through cloudevo-vpce-s3.
- **Evidence:** web hostname c508d5e66ff3 → f9d769cd28a0; Order #201 ($550.00) verified through the private path.

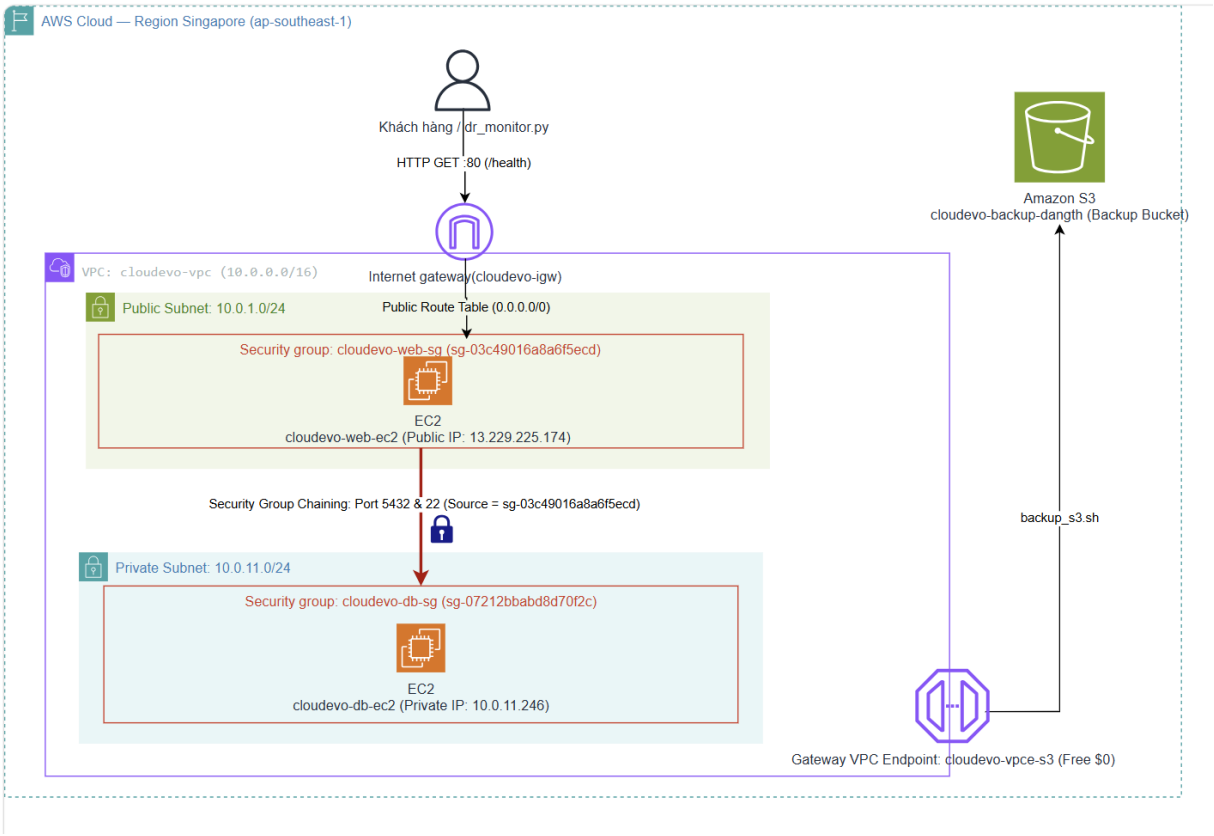


Figure 1.3: Scenario 3 — public web EC2 and private DB EC2 connected through security-group chaining.

Scenario 4: Web on EC2 + Amazon RDS for PostgreSQL (cloud-native)

At a glance | Deploy: 45 min (ClickOps) / ~11m 00s (User Data) / Terraform not measured | Cost: $36.22/month Single-AZ deployed ($57.96 Multi-AZ, priced only) | Measured RTO: 11.46 s (RDS reboot) | Data loss: 0 in the reboot test, PITR available | Best for: production-like workloads needing managed backups

Source code: scenarios/case4-ec2-web-rds | **Experiment report:** docs/Day4

- **Web tier** (cloudevo-web-rds-ec2): stateless EC2 t3.micro, 20 GB gp3, public subnet 10.0.1.0/24, public IPv4 13.212.204.91, container hostname e304be23ed60.
- **Database tier** (cloudevo-rds-postgres, resource ID db-KCTYRDD5IYTKGPM2M5K4DGE3G4): RDS PostgreSQL 16.10 on db.t4g.micro (Graviton2, 2 vCPU, 1 GiB), 20 GiB gp3, **Single-AZ as deployed**, in cloudevo-db-subnet-group (private subnets in 1a and 1b), PubliclyAccessible = false.
- **Encryption and recovery:** storage encrypted at rest with the AWS managed KMS key (aws/rds); automated backups (7-day retention) archive transaction logs for Point-in-Time Recovery (PITR). A Multi-AZ synchronous standby is an optional upgrade that was priced but not deployed or tested.
- **Alerting:** CloudWatch alarms (CPU > 80% and application HTTP 503) publish to an Amazon SNS topic (Ops-Alerts) that notifies the on-call engineer by email; defined in `terraform/monitoring.tf`.
- **Resilience in the app:** the Node.js connection pool times out after 2 s, the API answers HTTP 503 while the database is unavailable and reconnects automatically to the static RDS DNS endpoint. Measured `/health` latency to RDS: 2–28 ms.

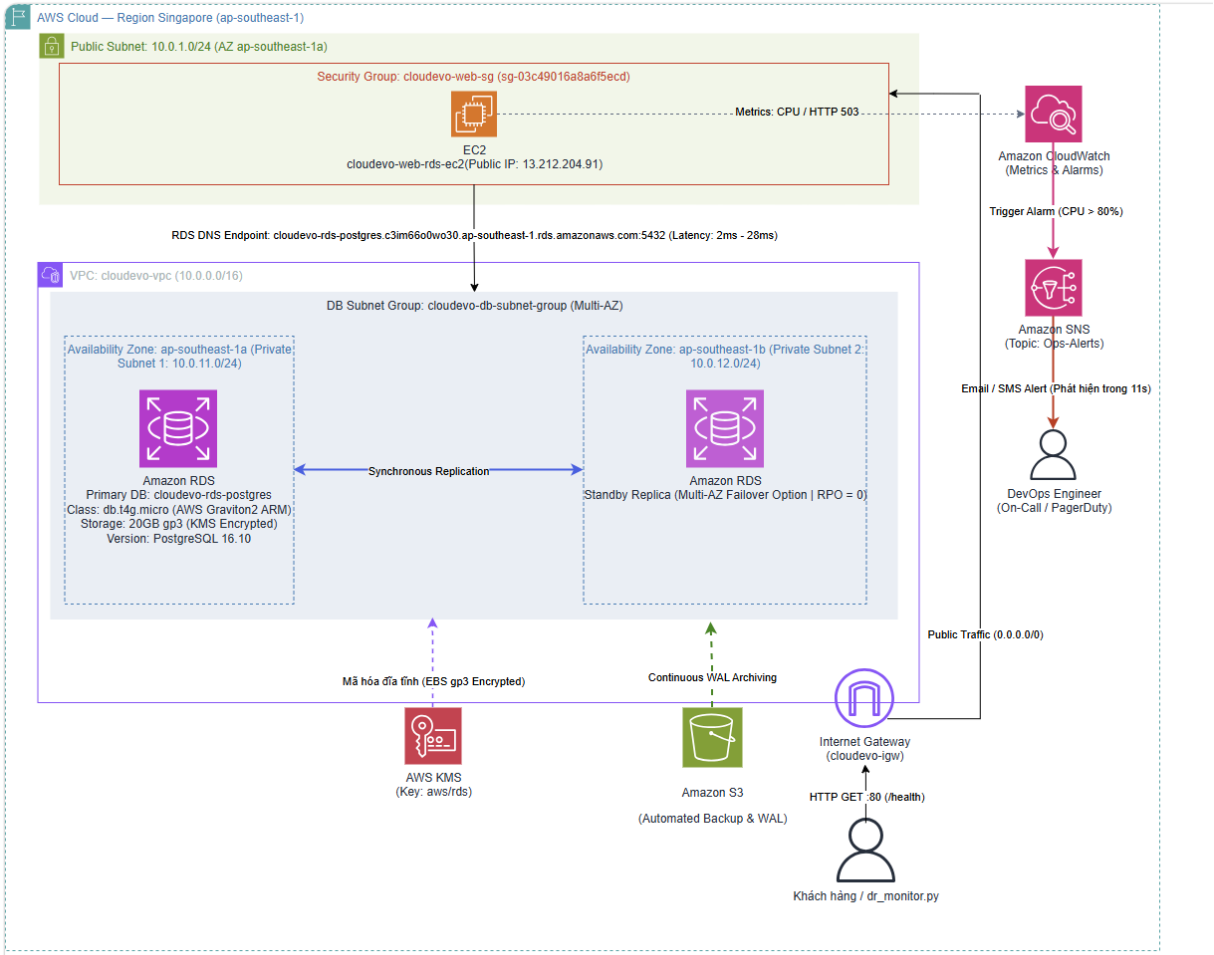


Figure 1.4: Scenario 4 — EC2 web tier + RDS PostgreSQL 16.10 with KMS encryption, WAL archiving and alerting. The standby replica drawn is the optional Multi-AZ configuration (priced in Section 4, not deployed in the lab).

3. How long does it take to host the application on AWS?

Table 2 — Deployment time by method

Scenario	Manual ClickOps (console + SSH)	EC2 User Data (semi-automated)	Terraform (fully automated)	User Data vs ClickOps	Terraform vs ClickOps
1: Stateless EC2	20.0 min (1,200 s)	2m 48s (168 s)	~1m 45s (105 s): VPC + SG + IAM + EC2	−86.0%	−91%
2: Web + local DB	35.0 min (2,100 s)	3m 15s (195 s)	**NOT MEASURED**	−90.7%	—
3: Web EC2 + DB EC2	50.0 min (3,000 s)	~5m 00s (300 s): two hosts, about 2m 30s each	**NOT MEASURED**	−90.0%	—
4: Web EC2 + RDS	45.0 min (2,700 s)	~11m 00s (660 s): RDS 8m 30s + web 2m 30s	**NOT MEASURED**	−75.6%	—

- **Method.** Every duration is a single run. ClickOps was self-timed from opening the console to a healthy `/health` response. User Data timings start at instance launch and exclude the one-time VPC/IAM setup; the Terraform timing for Scenario 1 includes it, so the Terraform figure is the more complete one.
- **Terraform run (Scenario 1).** `terraform apply -var="scenario_case=1"` created the full stack in about 1m 45s (about 40 s for network and IAM, then EC2 launch and Docker bootstrap in parallel). `terraform destroy` removed all 21 resources in about 45 s.
- **Why RDS takes longest.** Allocating Graviton2 compute, initializing KMS-encrypted gp3 storage and registering the DB subnet group takes about 8m 30s, while the web EC2 finishes bootstrapping in parallel in about 2m 30s.
- **Errors seen in manual runs:** overly permissive .pem permissions (SSH key rejected), forgotten security-group rules between tiers, and missing backup crontabs. Scripted builds removed these classes of mistakes.
- **Application releases (CI/CD).** A GitHub Actions run takes about 55 s: Docker build, container smoke test on `/health`, and Terraform format/syntax validation (Section 6, Step 5).

4. How much does it cost per month?

Pricing basis (on-demand, ap-southeast-1, 730 h/month, data transfer excluded):

- EC2 t3.micro: $0.0132/h = $9.64/month. EBS gp3: $0.096/GB-month. Public IPv4 address: $0.005/h = $3.65/month (charged since February 2024).
- RDS db.t4g.micro for PostgreSQL: $0.025/h Single-AZ = $18.25/month; $0.051/h Multi-AZ = $37.23/month. RDS gp3 storage: $0.138/GB-month Single-AZ ($0.276 Multi-AZ); automated backup storage is free up to the provisioned database size.
- S3 and snapshot storage for Scenarios 2–3 are small estimates (~$0.86 and ~$1.33/month).
- EC2, RDS instance and NAT Gateway rates were checked against public price lists in October 2026. EBS gp3 and RDS gp3 storage are list rates that should be re-confirmed in the AWS Pricing Calculator before quoting a customer.

Table 3 — Monthly on-demand cost breakdown (USD)

Component	Scenario 1	Scenario 2	Scenario 3	Scenario 4A RDS Single-AZ	Scenario 4B RDS Multi-AZ
Web EC2 t3.micro	9.64	9.64	9.64	9.64	9.64
DB compute (EC2 or RDS db.t4g.micro)	0.00 (in memory)	0.00 (co-located)	9.64 (private EC2)	18.25	37.23 (primary + standby)
Block / database storage (gp3)	1.92 (20 GB EBS)	2.40 (25 GB EBS)	3.84 (2×20 GB EBS)	4.68 (20 GB EBS + 20 GB RDS: 1.92 + 2.76)	7.44 (1.92 + 5.52)
Public IPv4 (web only)	3.65	3.65	3.65	3.65	3.65
Backups and snapshots	0.00	~0.86	~1.33	0.00 (free up to DB size)	0.00
VPC, IGW, S3 Gateway Endpoint	0.00	0.00	0.00	0.00	0.00
Total	**15.21**	**16.55**	**28.10**	**36.22**	**57.96**

CloudWatch alarms (about $0.10 per standard alarm per month) and SNS email notifications are not included above; with two alarms they add well under $1/month.

FinOps design choice: no NAT Gateway

A NAT Gateway in ap-southeast-1 costs $0.059/h = **$43.07/month** before per-GB data processing (and needs a public IPv4 address). Adding one to Scenario 3 would lift its bill from $28.10 to at least $71.17 (+153%). The design avoids it: backups use the free S3 Gateway Endpoint (no hourly or per-GB charge), and the private DB host needed Internet access only once, at first boot.

Indicative savings with commitments or Spot

Table 4 — Compute lines only (replace the matching line in Table 3 to get a scenario total)

Compute line	On-demand	1-year commitment	3-year commitment	Spot
Web EC2 t3.micro	$9.64	~$6.94 (Compute Savings Plan, ~28% off)	~$4.63 (~52% off)	~$4.09 (about $0.0056/h; can be interrupted)
RDS db.t4g.micro (Single-AZ)	$18.25	~$14 (Reserved Instance)	~$11 (Reserved Instance)	Not available

Discount levels are indicative public-list values for burstable instances in ap-southeast-1 and vary by term and payment option; confirm them before committing.

Cost guardrail

An AWS Budget (`setup_budget.sh`) caps the benchmark at $100/month and emails **tranhaidang.devops@gmail.com** at 80% of actual spend and at 100% of forecast spend.

5. If an outage occurs: how long to recover (RTO) and how much data is lost (RPO)?

All results below tagged **MEASURED** were captured under live failure drills with `scripts/dr_monitor.py` (1-second polling of `/health`) and timestamped transactions. Clock times are Vietnam time (UTC+7) unless marked UTC; the application UI shows UTC.

Table 5 — Recovery matrix by failure type (summary of Tables 6 and 7)

Scenario	Resource stress	Instance termination	Database outage / reboot / loss	Data loss (RPO)	Fault tolerance (qualitative)
1: Stateless EC2	Not tested	449.48 s (manual replacement)	n/a (no database)	100% of in-memory data	Low: one instance, volatile state
2: Web + local DB	34.96 s freeze (DB CPU/RAM saturation)	Not tested (5–8 min **ESTIMATE**)	82.49 s (containers + volume wiped, S3 restore)	Up to 1 h (Order #103 lost)	Very low: web and DB share one host
3: Web EC2 + DB EC2	Not tested	Web: 86.05 s. DB EC2: Not tested (15–20 min **ESTIMATE**)	44.07 s (PostgreSQL stopped, S3 restore)	Web: 0. DB: up to 1 h	Medium: tiers isolated, one AZ, DB relies on hourly dumps
4: Web EC2 + RDS	Not tested	Web: Not tested	11.46 s (RDS reboot); PITR restore ready in ~8–10 min	0 in reboot test; PITR < 5 min per AWS design (**ESTIMATE**)	Higher: managed DB reconnects automatically; AZ failure and Multi-AZ failover not tested

Table 6 — Measured failure drills

Test	Failure injected	Outage (failed probes)	Data outcome	Notes
1A	Scenario 1: EC2 instance terminated	449.48 s (150)	100% of in-memory data lost	**MEASURED** Manual recovery: ~15 s detection, ~1.7 min console launch, ~45 s boot, ~4.4 min User Data build, ~19 s Elastic IP re-association. Host fe5dc1f64f31 → 363732ec3ca3.
2A	Scenario 2: stress test saturating PostgreSQL CPU/RAM on the shared host	34.96 s (12)	None observed	**MEASURED** DB starved the Node.js event loop and Nginx: single point of failure. Host 75da0efaeb9f (no restart).
2B	Scenario 2: containers and volume wiped, restore from the latest S3 dump (restore_s3.sh)	82.49 s (28)	Data written after the last hourly backup is lost (see 2C)	**MEASURED** 19:59:22 → 20:00:47. New host 5ede407bab53.
2C	Scenario 2: RPO check (separate run)	—	Order #1 present after restore; Order #103 ($999.00, created after the last backup) lost	**MEASURED** Hosts 9db872eb2c3f → fa46b20a328f. Demonstrates the 1-hour cron RPO window.
3A	Scenario 3: web tier killed	86.05 s (29)	0 bytes lost (Order #201, $550.00 intact)	**MEASURED** 07:56:41 → 07:58:08. Host c508d5e66ff3 → f9d769cd28a0. DB on 10.0.11.246 unaffected.
3B	Scenario 3: PostgreSQL stopped on the DB EC2, restored from S3	44.07 s (39)	Up to 1 h (hourly dump)	**MEASURED** Web tier stayed up and returned HTTP 503. 08:42:39 → 08:43:24.
4A	Scenario 4: RDS instance reboot (Single-AZ)	11.46 s (10)	0 bytes lost (Order #401, $1,250.00 intact)	**MEASURED** Web container unchanged (e304be23ed60), automatic reconnect, no human action. 12:17:41 → 12:17:54.
4B	Scenario 4: Point-in-Time Recovery to 12:17:42	New instance cloudevo-rds-pitr-restore Available in ~8–10 min	Restore point chosen 1 s after test 4A	**MEASURED** Recovery point per AWS design: latest restorable time typically within 5 min (**ESTIMATE**).

Table 7 — Figures that were not measured in the lab

Item	Value	Status
Scenario 2: full EC2 replacement (82.49 s was a container/volume wipe, not a new host)	5–8 min	**ESTIMATE**
Scenario 3: DB EC2 replacement from an EBS snapshot	15–20 min	**ESTIMATE**
Scenario 4: Multi-AZ failover	Typically 60–120 s (AWS-documented range)	**NOT MEASURED**: Multi-AZ was not deployed
Scenarios 3 and 4: web/DB resource isolation under CPU/RAM stress	Expected by design (no cross-impact)	**NOT MEASURED**: only Scenario 2 was stress-tested
Scenario 4: web EC2 termination	Closest measured value: 86.05 s (Scenario 3, test 3A)	**NOT MEASURED**
Alert delivery time of the CloudWatch/SNS alarms	—	**NOT MEASURED**

- **Reading the matrix.** Scenario 1 is for disposable, stateless front ends. Scenario 2 trades a 34.96 s freeze risk and up to one hour of data loss for the lowest cost with persistence. Scenario 3 isolates the tiers (web failure RPO = 0) but still depends on hourly dumps for the database. Scenario 4 adds managed recovery: reconnection in 11.46 s with no data loss in the reboot test and second-level PITR.

6. Step-by-step implementation guide (from zero to live)

Prerequisites: an AWS account, AWS CLI v2, Terraform ≥ 1.5, Docker, Git and Python 3. The deployment is split into eight steps.

Step 0: Dedicated AWS CLI profile

```
aws configure --profile cloudevo-admin       # or: aws configure sso (preferred: no long-lived keys)
aws sts get-caller-identity --profile cloudevo-admin
export AWS_PROFILE=cloudevo-admin
export AWS_REGION=ap-southeast-1
```

Step 1: Budget and billing alert (FinOps guardrail)

```
# edit ALERT_EMAIL=tranhaidang.devops@gmail.com inside the script, then:
bash setup_budget.sh
# creates the monthly budget "CloudEvo-Benchmark-Monthly-Guardrail" (USD 100)
# alerts: 80% of actual spend and 100% of forecast spend
```

Step 2: Multi-AZ network foundation (terraform/vpc.tf)

Execution: part of the single Terraform run in Step 4 (network and IAM take about 40 s). **Resources created:** VPC, Internet Gateway, 4 subnets, route tables and associations, S3 Gateway Endpoint.

- VPC cloudevo-vpc 10.0.0.0/16 with DNS support and DNS hostnames enabled, and Internet Gateway cloudevo-igw.
- Public subnets 10.0.1.0/24 (1a) and 10.0.2.0/24 (1b) with public IP on launch; private subnets 10.0.11.0/24 (1a) and 10.0.12.0/24 (1b).
- Public route table 0.0.0.0/0 → IGW; private route tables have no Internet route. S3 Gateway VPC Endpoint cloudevo-vpce-s3 is attached to the route tables.

Step 3: Security groups, IAM and backup bucket (security_groups.tf, compute_ec2.tf)

Execution: part of the single Terraform run in Step 4. **Resources created:** web and DB security groups, IAM role, policy and instance profile, optional S3 backup bucket.

- cloudevo-web-sg: inbound TCP 80 and 22 (SSH is open to the Internet in the lab only; restrict it or use SSM Session Manager in production). cloudevo-db-sg: inbound TCP 5432 and 22 only from the web security group.
- IAM role and instance profile for EC2 with the least-privilege S3 policy shown in Section 2; the backup bucket is created by Terraform when `create_s3_bucket = true`.

Step 4: Parameterized deployment with Terraform (terraform/)

Execution time: ~1m 45s for Scenario 1 (measured). **Resources created:** 21 for Scenario 1 (the same 21 removed in Step 7).

Seven files (main.tf, variables.tf, vpc.tf, security_groups.tf, compute_ec2.tf, rds.tf, outputs.tf) plus monitoring.tf are driven by one variable, `scenario_case` (1–4). It selects the database target (local, container, private EC2 or RDS endpoint) and creates the DB EC2 (case 3) or the RDS instance (case 4) through `count` conditions.

```
cd terraform/
terraform init && terraform fmt -check && terraform validate
terraform apply -var="scenario_case=1" -auto-approve
# Outputs:
#   active_scenario_case = "Scenario Case 1"
#   db_connection_target = "localhost"
#   health_check_url     = "http://54.254.173.180/health"
#   vpc_id               = "vpc-0b501be8f24024987"
#   web_ec2_public_ip    = "54.254.173.180"
curl -s http://54.254.173.180/health
# {"status":"UP","scenario_case":1,"provisioned_by":"TerraformIaC","mode":"memory", ... "hostname":"11d0d4004004"}
```

For Scenarios 2–4 the full application is deployed by the scripts in `scenarios/` (user_data_case2.sh, setup_db_ec2.sh + setup_web_ec2.sh, setup_web_rds.sh). Running these scripts from Terraform is open work (Section 7).

Step 5: CI/CD with GitHub Actions (.github/workflows/deploy.yml)

Execution time: ~55 s per run (recorded run). **Actions:** Docker build, container smoke test on /health, Terraform format and validation.

Every push or pull request to main, and every manual dispatch, runs two parallel jobs:

- **Build Docker and verify /health:** builds `cloudevo-tracker:<commit sha>` from ./app, starts a container with STORAGE_MODE=memory, waits 4 s and requires `curl -f http://127.0.0.1:3000/health` to succeed; on push it logs in to GHCR and pushes the `:latest` tag.
- **Terraform format and syntax validation:** Terraform 1.7.5, `terraform fmt -check`, `terraform init -backend=false`, `terraform validate`.
- A recorded run finished in 55 s (jobs run in parallel).

Step 6: Health probing and failure drills (scripts/dr_monitor.py)

Execution: runs continuously during each drill. **Output:** one line per probe; the outage duration (RTO) is printed on recovery.

```
python3 scripts/dr_monitor.py 13.212.204.91        # URL or IP; "/health" is appended automatically
# polls every second (2 s timeout), prints each failed probe, and on recovery prints
# the total outage duration (the measured RTO) and the new container hostname
# example steady-state line:
# [12:17:38] [UP] HTTP 200 | Mode: postgres | DB: True | Latency: 139ms | Host: e304be23ed60
```

- Drills used: terminate the instance (1A); stress-ng on PostgreSQL (2A); wipe containers and volume, then restore from S3 (2B, 2C); kill the web tier (3A); stop PostgreSQL and restore from S3 (3B); reboot RDS (4A); PITR restore (4B).
- `rds_failover_test.sh` uses `--force-failover`, which is valid only for Multi-AZ instances; on the Single-AZ lab instance a plain reboot was used.

Step 7: Zero-residue teardown

Execution time: ~45 s for the 21 resources of Scenario 1 (measured).

```
terraform destroy -var="scenario_case=1" -auto-approve
# aws_vpc.main: Destruction complete after 1s
# Destroy complete! Resources: 21 destroyed.
```

Terraform removes only what it created. Resources built by hand (ClickOps and User Data EC2 hosts, RDS, snapshots, Elastic IPs, test buckets) must be deleted separately; they were removed after the logs were captured.

7. Limitations and next steps

Area	Current state	Next step
Terraform for Scenarios 2–4	Network, security groups, IAM, S3 bucket, EC2 and RDS resources are codified, but only Scenario 1 was applied and timed. The Terraform web bootstrap is a minimal placeholder health service, not the full Express/PostgreSQL stack.	Run the scenario scripts through user_data / templatefile(), then apply and time Scenarios 2–4.
Repeatability	Every timing and drill is a single run.	Repeat at least three times and report the median.
RDS Multi-AZ	Priced, not deployed or tested.	Set rds_multi_az = true, run a reboot with failover, measure RTO.
Alerting	CloudWatch alarms and the SNS topic are defined in monitoring.tf; alert delivery time was **not measured**.	Trigger a test alarm and record the notification delay.
Pipeline security	CI builds, smoke-tests and validates; no vulnerability scanning yet.	Add a container scan (for example Trivy) and an IaC scan.
Lab shortcuts	SSH open to 0.0.0.0/0; demo database passwords in scripts; local Terraform state.	Use SSM Session Manager or an admin CIDR; AWS Secrets Manager / random_password; S3 backend with encryption and state locking.
Edge, HTTPS and self-healing	Lab traffic is plain HTTP to a single instance per tier; there is no load balancer, Auto Scaling Group or Secrets Manager.	For production add TLS (ACM with ALB or CloudFront), an Auto Scaling Group for instance self-healing, AWS Secrets Manager for credentials and SSM Session Manager instead of SSH; re-price the result (a NAT Gateway alone is $43.07/month).

8. Recommendations

- **Scenario 1 ($15.21/month):** landing pages, stateless APIs and demos. Never for transactional data: a terminated instance loses all state.
- **Scenario 2 ($16.55/month):** internal tools and low-budget MVPs that can accept up to one hour of data loss and a CPU/RAM contention risk (34.96 s freeze measured).
- **Scenario 3 ($28.10/month):** when web and database must scale or be tuned separately (custom extensions, OS-level control). Web failures lose no data; DB failures still depend on hourly dumps.

- **Scenario 4 ($36.22/month Single-AZ, $57.96 Multi-AZ):** commercial workloads that need managed backups and PITR. Compared with Scenario 3 it costs +$8.12/month (+29%) and gives managed patching, encryption at rest, 11.46 s automatic reconnection and second-level PITR. For production, validate Multi-AZ failover, move Terraform state to an encrypted S3 backend and add cost checks to CI.

Evidence index (repository)

Location	Content
docs/Day1 – docs/Day4	Daily experiment reports with screenshots: Scenario 1 (Day 1), Scenario 2 (Day 2), Scenario 3 (Day 3), Scenario 4 (Day 4).
docs/Day5	Terraform automation, CI/CD run and FinOps teardown evidence.
scripts/dr_monitor.py	1-second health probe used for every RTO measurement.
scenarios/, terraform/, .github/workflows/	Per-scenario deployment scripts, Terraform code and the CI pipeline.